Zadanie Cząstkowe 3: Implementacja w Kubernetes i Sprawozdanie z Realizacji Testów

Przedmiot: Bezpieczeństwo CI/CD / Zaawansowane Środowiska Chmurowe i Konteneryzacja
Temat: 3 – System zbierania metryk i monitoringu (Prometheus + Grafana + własny eksporter w Pythonie) **Data wykonania zrzutów i testów:** 2026-11-05 (Minikube v1.35 + GitHub Actions)

Linki do projektu (repozytorium publiczne)

Zasób	Adres
B. Repozytorium z manifestami	https://github.com/Shengan01/Inference-Server
Manifesty Kubernetes (`base` , `overlays` , `addons` , `gitops` , `tests` , `examples`)	https://github.com/Shengan01/Inference-Server/tree/main/k8s
README repozytorium	https://github.com/Shengan01/Inference-Server/blob/main/README.md
GitHub Actions (potok CI/CD, wszystkie przebiegi)	https://github.com/Shengan01/Inference-Server/actions
Definicja potoku	https://github.com/Shengan01/Inference-Server/blob/main/.github/workflows/ci-cd.yaml
Przykładowy przebieg z raportem testów (artefakt `test-report`)	https://github.com/Shengan01/Inference-Server/actions/runs/41239646305
Obraz kontenera (GitHub Container Registry)	https://github.com/Shengan01/Inference-Server/pkgs/container/mcp-agent
Instrukcja uruchomienia i testów	https://github.com/Shengan01/Inference-Server/blob/main/docs/INSTRUKCJA_URUCHOMIENIA_I_TESTOW.md

Zawartość sprawozdania. Część **A** (rozdz. 1–3): potwierdzenie wdrożenia w klastrze (polecenia + zrzuty), prezentacja testów z Zadania 2 (cel, polecenia, zrzuty, wyniki) oraz komentarz do funkcjonalności. Część **B** (rozdz. 4): link do repozytorium z manifestami i opis jego zawartości. Zrzuty z terminala przedstawiają rzeczywisty wynik poleceń uruchomionych na klastrze (generowane skryptem `scripts/capture_evidence.py`), a zrzuty interfejsów (Prometheus, Grafana, Alertmanager, GitHub Actions) są zrzutami z przeglądarki.

Część A

1. Potwierdzenie wdrożenia w klastrze Kubernetes

Środowisko: jednowęzłowy klaster Minikube (sterownik Docker, Kubernetes v1.35), namespace `inference-monitoring` . System wdrożono dwiema równoważnymi ścieżkami: skryptem lokalnym oraz

GitOps (ArgoCD synchronizuje klaster z repozytorium).

```
./scripts/deploy_minikube.sh cpu-demo                          # wdrożenie lokalne (Kustomize,
overlay local-cpu)
kubectl apply -f k8s/gitops/argocd-application.yaml        # wdrożenie GitOps (overlay cpu-demo
z gałęzi main)
```

1.1. Mapa wymaganych obiektów

Obiekt	Konfiguracja w projekcie	Manifest	Zrzut
StatefulSet	`prometheus`: stała tożsamość, `volumeClaimTemplates` (2Gi), retencja 15 dni, probes, `securityContext`	`k8s/base/prometheus/statefulset.yaml`	1.3
Deployment	`grafana`, `alertmanager`, `mcp-agent` (eksporter), `triton-server` (mock); probes, strategia `Recreate` dla Grafany (PVC RWO)	`k8s/base/*/deployment.yaml`, `k8s/overlays/cpu-demo/`	1.3
Service	5 × `ClusterIP` (Prometheus, Alertmanager, Grafana, Triton, MCP Agent), nazwane porty	`k8s/base/*/service.yaml`	1.4
Ingress	NGINX, TLS, host `inference.local`; ścieżki `/grafana`, `/v2`, `/mcp` (rewrite)	`k8s/base/ingress/ingress.yaml`	1.4
PVC	`prometheus-storage-prometheus-0` 2Gi (z szablonu StatefulSet), `grafana-storage` 1Gi	`k8s/base/grafana/pvc.yaml`, StatefulSet	1.5
ConfigMap	konfiguracja Prometheusa i Alertmanagera, reguły alertów, datasource i dashboard Grafany, konfiguracja modelu	`k8s/base/*/configmap*.yaml`	1.6
Secret	`grafana-secret` (Opaque), `inference-tls` (`kubernetes.io/tls`); tworzone skryptem, bez wartości w Git; wzór w repozytorium	`scripts/bootstrap_secrets.sh`, `k8s/examples/secrets.example.yaml`	1.6
NetworkPolicy	`default-deny-all` + 6 reguł dla każdej pary komunikujących się usług	`k8s/base/network-policy.yaml`	1.7
Requests i limits	jawne `requests` i `limits` CPU/pamięć w każdym kontenerze (także init-container)	manifesty workloadów	1.8
ResourceQuota	4 CPU / 8Gi (requests), 8 CPU / 16Gi (limits), 5 PVC, 15 podów	`k8s/base/resource-quota.yaml`	1.8
LimitRange	domyślne requests/limits kontenera oraz zakres 500Mi–10Gi dla PVC	`k8s/base/resource-quota.yaml`	1.8

1.2. Klaster i namespace

```
kubectl get nodes -o wide && kubectl get namespaces
```

Klaster i namespace

```
$ kubectl get nodes -o wide && kubectl get namespaces
NAME       STATUS   ROLES           AGE    VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE                         KERNEL-VERSION           CONTAINER-RUNTIME
minikube   Ready    control-plane   228d   v1.35.1   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm)   7.2.6-76070206-generic   docker://29.2.1
NAME                   STATUS   AGE
argocd                 Active   5d
default                Active   228d
inference-monitoring   Active   21d
ingress-nginx          Active   21d
kube-node-lease        Active   228d
kube-public            Active   228d
kube-system            Active   228d
monitoring             Active   5d
```

1.3. Deployment i StatefulSet

```
kubectl -n inference-monitoring get deployments,statefulsets,pods
```

Deployment / StatefulSet / Pod

```
$ kubectl -n inference-monitoring get deployments,statefulsets,pods
NAME                              READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/alertmanager      1/1     1            1           5d3h
deployment.apps/grafana           1/1     1            1           21d
deployment.apps/mcp-agent         1/1     1            1           21d
deployment.apps/triton-server     1/1     1            1           21d

NAME                             READY   AGE
statefulset.apps/prometheus      1/1     21d

NAME                                  READY   STATUS    RESTARTS        AGE
pod/alertmanager-7448b9bdc-kjdrj      1/1     Running   2 (4d23h ago)   5d3h
pod/grafana-db86b4cc6-bdwvp           1/1     Running   0               15m
pod/mcp-agent-5958656d45-mz77k        1/1     Running   0               31m
pod/prometheus-0                      1/1     Running   2 (4d23h ago)   5d3h
pod/triton-server-7c8965976d-cw5qf    1/1     Running   0               17m
```

Komentarz: Prometheus działa jako StatefulSet (dane szeregów czasowych na trwałym wolumenie), pozostałe komponenty jako Deployment. Wszystkie pody są w stanie `Running 1/1`. Restarty wynikają z zatrzymania i ponownego uruchomienia Minikube przed wykonaniem zrzutów.

1.4. Service i Ingress

```
kubectl -n inference-monitoring get services,ingress
IP=$(minikube ip); curl -sk --resolve inference.local:443:$IP
https://inference.local/grafana/api/health
```

Service i Ingress

```
$ kubectl -n inference-monitoring get services,ingress
NAME                                TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                      AGE
service/alertmanager-service        ClusterIP   10.101.176.4     <none>        9093/TCP                     15m
service/grafana-service             ClusterIP   10.104.160.162   <none>        3000/TCP                     21d
service/mcp-agent-service           ClusterIP   10.108.212.120   <none>        5000/TCP                     21d
service/prometheus-service          ClusterIP   10.105.191.162   <none>        9090/TCP                     21d
service/triton-service              ClusterIP   10.102.129.255   <none>        8000/TCP,8001/TCP,8002/TCP   21d

NAME                                                              CLASS   HOSTS             ADDRESS        PORTS     AGE
ingress.networking.k8s.io/inference-monitoring-ingress            nginx   inference.local   192.168.49.2   80, 443   21d
ingress.networking.k8s.io/inference-monitoring-mcp-ingress        nginx   inference.local   192.168.49.2   80, 443   5d3h
```

Dostęp zewnętrzny przez Ingress (TLS, kody odpowiedzi i przekierowanie HTTP → HTTPS):

Dostęp przez Ingress (TLS, przekierowanie HTTP -> HTTPS)

```
$ IP=$(minikube ip); for p in grafana/api/health v2/health/ready mcp/health; do echo "https://inference.local/$p -> $(curl -sk --resolve inference.local:443:$IP https://inference.local/$p -o
   /dev/null -w '%{http_code}')"; done; echo "http://inference.local/grafana/ -> $(curl -s --resolve inference.local:80:$IP http://inference.local/grafana/ -o /dev/null -w '%{http_code}') (p
  rzekierowanie na HTTPS)"
https://inference.local/grafana/api/health -> 200
https://inference.local/v2/health/ready -> 200
https://inference.local/mcp/health -> 200
http://inference.local/grafana/ -> 308 (przekierowanie na HTTPS)
```

1.5. PersistentVolumeClaim

```
kubectl -n inference-monitoring get pvc && kubectl get pv
```

PersistentVolumeClaim i PersistentVolume

```
$ kubectl -n inference-monitoring get pvc && kubectl get pv
NAME                          STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
grafana-storage               Bound    pvc-446cee9d-995d-447e-8505-e5718fce22aa   1Gi        RWO            standard       <unset>                 5d3h
prometheus-storage-prometheus-0   Bound    pvc-cf38eacc-a090-4de5-a5a4-55ae2953ebb0   2Gi        RWO            standard       <unset>                 21d
NAME                                       CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM                                                                     STORAGECLASS   VOLUMEATTR
  IBUTESCLASS   REASON   AGE
pvc-2107fb4e-4e0f-444a-9449-23a233828c73   2Gi        RWO            Delete           Bound    default/guinea-pig-db-pvc                                                 standard       <unset>
                         228d
pvc-446cee9d-995d-447e-8505-e5718fce22aa   1Gi        RWO            Delete           Bound    inference-monitoring/grafana-storage                                      standard       <unset>
                         5d3h
pvc-9e3d6617-d3b2-43a2-a17c-1d5c6ed05ceb   2Gi        RWO            Delete           Bound    monitoring/prometheus-kps-prometheus-db-prometheus-kps-prometheus-0       standard       <unset>
                         5d
pvc-cf38eacc-a090-4de5-a5a4-55ae2953ebb0   2Gi        RWO            Delete           Bound    inference-monitoring/prometheus-storage-prometheus-0                      standard       <unset>
                         21d
```

1.6. ConfigMap i Secret

```
kubectl -n inference-monitoring get configmaps,secrets
```

ConfigMap i Secret (nazwy i typy, bez wartości)

```
$ kubectl -n inference-monitoring get configmaps,secrets
NAME                                 DATA   AGE
configmap/alertmanager-config        1      5d3h
configmap/grafana-dashboards         2      21d
configmap/grafana-datasources        1      21d
configmap/kube-root-ca.crt           1      21d
configmap/loadgen-script             1      5d1h
configmap/mock-triton-script         1      21d
configmap/prometheus-alerting-rules  1      21d
configmap/prometheus-config          1      21d
configmap/triton-model-config        1      21d

NAME                    TYPE                DATA   AGE
secret/grafana-secret   Opaque              2      21d
secret/inference-tls    kubernetes.io/tls   2      5d3h
```

Komentarz: wartości sekretów celowo nie są pokazywane ani przechowywane w Git. `grafana-secret` zawiera losowe hasło administratora, `inference-tls` certyfikat self-signed dla `inference.local`; oba tworzy `scripts/bootstrap_secrets.sh`. Wzór obiektów (z wartościami zastępczymi) znajduje się w `k8s/examples/secrets.example.yaml`.

1.7. NetworkPolicy

```
kubectl -n inference-monitoring get networkpolicies
```

NetworkPolicy

```
$ kubectl -n inference-monitoring get networkpolicies
NAME                              POD-SELECTOR                                          AGE
allow-alertmanager                app.kubernetes.io/name=alertmanager                   5d3h
allow-grafana                     app.kubernetes.io/name=grafana                        21d
allow-kube-dns                    <none>                                                21d
allow-loadgen-to-prometheus       app.kubernetes.io/name=prometheus                     5d1h
allow-mcp-agent                   app.kubernetes.io/name=mcp-agent                      21d
allow-operator-prometheus-scrape  app.kubernetes.io/name in (mcp-agent,triton-server)   5d
allow-prometheus-scrape           app.kubernetes.io/name=prometheus                     21d
allow-triton-server               app.kubernetes.io/name=triton-server                  21d
default-deny-all                  <none>                                                21d
loadgen-egress                    app.kubernetes.io/name=loadgen                        5d1h
```

Komentarz: `default-deny-all` blokuje cały ruch w namespace, a pozostałe polityki otwierają wyłącznie wymagane połączenia (Prometheus → Triton/agent/Alertmanager, Grafana → Prometheus, agent → Prometheus/Triton, ruch z Ingress do usług, DNS). Polityki `loadgen-*` pochodzą z Jobu testowego (rozdz. 2).

1.8. Requests, limits, ResourceQuota i LimitRange

```
kubectl -n inference-monitoring get pods -o custom-
columns=POD:.metadata.name,CONTAINER:.spec.containers[*].name,CPU_REQ:.spec.containers[*].resou
rces.requests.cpu,CPU_LIM:.spec.containers[*].resources.limits.cpu,MEM_REQ:.spec.containers[*].
resources.requests.memory,MEM_LIM:.spec.containers[*].resources.limits.memory
kubectl -n inference-monitoring describe resourcequota inference-monitoring-quota
kubectl -n inference-monitoring describe limitrange inference-monitoring-limits
```

Requests i limits każdego kontenera

```
$ kubectl -n inference-monitoring get pods -o custom-columns=POD:.metadata.name,CONTAINER:.spec.containers[*].name,CPU_REQ:.spec.containers[*].resources.requests.cpu,CPU_LIM:.spec.containers
  [*].resources.limits.cpu,MEM_REQ:.spec.containers[*].resources.requests.memory,MEM_LIM:.spec.containers[*].resources.limits.memory
POD                              CONTAINER           CPU_REQ   CPU_LIM   MEM_REQ   MEM_LIM
alertmanager-7448b9bdc-kjdrj     alertmanager        50m       200m      64Mi      128Mi
grafana-db86b4cc6-bdwvp          grafana             100m      500m      128Mi     512Mi
mcp-agent-5958656d45-mz77k       mcp-agent           100m      500m      128Mi     512Mi
mcp-agent-9cd96c76c-z9rkb        mcp-agent           100m      500m      128Mi     512Mi
prometheus-0                     prometheus          200m      1         256Mi     1Gi
triton-server-7c8965976d-cw5qf   mock-triton-server  100m      500m      64Mi      256Mi
```

ResourceQuota

```
$ kubectl -n inference-monitoring describe resourcequota inference-monitoring-quota
Name:                   inference-monitoring-quota
Namespace:              inference-monitoring
Resource                Used    Hard
--------                ----    ----
limits.cpu              2700m   8
limits.memory           2432Mi  16Gi
persistentvolumeclaims  2       5
pods                    5       15
requests.cpu            550m    4
requests.memory         640Mi   8Gi
```

LimitRange

```
$ kubectl -n inference-monitoring describe limitrange inference-monitoring-limits
Name:                   inference-monitoring-limits
Namespace:              inference-monitoring
Type                    Resource  Min    Max   Default Request  Default Limit  Max Limit/Request Ratio
----                    --------  ---    ---   ---------------  -------------  -----------------------
Container               cpu       -      -     100m             1              -
Container               memory    -      -     256Mi            1Gi            -
PersistentVolumeClaim   storage   500Mi  10Gi  -                -              -
```

1.9. GitOps i profil operatora

```
kubectl -n argocd get applications && kubectl -n monitoring get pods
```

GitOps: aplikacje ArgoCD i profil operatora

```
$ kubectl -n argocd get applications && kubectl -n monitoring get pods
NAME                      SYNC STATUS   HEALTH STATUS
inference-monitoring-app  Synced        Progressing
kube-prometheus-stack     Synced        Healthy
operator-stack-resources  Synced        Healthy
NAME                                                     READY   STATUS    RESTARTS        AGE
alertmanager-kps-alertmanager-0                          2/2     Running   2 (4d23h ago)   5d
kps-operator-f4ccdcbf5-hrn9r                             1/1     Running   2 (31m ago)     5d
kube-prometheus-stack-grafana-69d8d49768-gp7tx           3/3     Running   3 (4d23h ago)   5d
kube-prometheus-stack-kube-state-metrics-6975ffc467-njf64  1/1   Running   2 (31m ago)     5d
kube-prometheus-stack-prometheus-node-exporter-g89rb     1/1     Running   1 (4d23h ago)   5d
prometheus-kps-prometheus-0                              2/2     Running   2 (4d23h ago)   5d
```

1.10. Dodatkowe obiekty i zasoby

Obiekt	Uzasadnienie
`ServiceAccount` ×5, `ClusterRole` + `ClusterRoleBinding`	RBAC najmniejszych uprawnień: osobne konta dla każdego komponentu, Prometheus tylko do odczytu (bez dostępu do Secrets)
`Namespace` z etykietami Pod Security	`enforce: baseline`, `audit` / `warn: restricted`
`securityContext` + probes	`runAsNonRoot`, `drop: ALL`, `seccomp: RuntimeDefault`, `readinessProbe` / `livenessProbe` w każdym komponencie
`Job` + `NetworkPolicy` testowe	generator obciążenia działa wewnątrz klastra (bez narzutu `port-forward`)
`ServiceMonitor` ×2, `PrometheusRule` (CRD)	profil Prometheus Operator, wykrywanie celów przez zasoby Kubernetes
Helm (`kube-prometheus-stack`)	gotowy stos operatora wdrażany z własnymi wartościami
ArgoCD `Application` ×3	GitOps: automatyczna synchronizacja, `prune`, `selfHeal`
Kustomize (`base`, 5 overlayów)	jedna baza, profile `cpu-demo` / `gpu` / `local-*`
GitHub Actions + Trivy + kubeconform	walidacja i skan manifestów oraz obrazu, test na `kind`, automatyczne wdrożenie obrazu

2. Prezentacja testów

Testy odpowiadają procedurze zaplanowanej w Zadaniu 2 (`docs/ZADANIE_2_OPIS_I_ARCHITEKTURA.md`, rozdz. 3, testy 1–8). Poniżej: cel (lub odniesienie), użyte polecenia, zrzut i wynik.

Test 1: integralność wdrożenia i quota

Cel: wszystkie obciążenia działają, PVC są `Bound`, zużycie mieści się w quota (Zadanie 2, Test 1).

```
kubectl -n inference-monitoring get pods,pvc
kubectl -n inference-monitoring describe resourcequota inference-monitoring-quota | tail -9
```

Test 1: integralność wdrożenia i quota

```
$ kubectl -n inference-monitoring get pods,pvc && kubectl -n inference-monitoring describe resourcequota inference-monitoring-quota | tail -9
NAME                                READY   STATUS     RESTARTS        AGE
pod/alertmanager-7448b9bdc-kjdrj    1/1     Running    2 (4d23h ago)   5d3h
pod/grafana-db86b4cc6-bdwvp         1/1     Running    0               15m
pod/mcp-agent-5958656d45-mz77k      1/1     Running    0               31m
pod/mcp-agent-9cd96c76c-z9rkb       0/1     Init:0/1   0               1s
pod/prometheus-0                    1/1     Running    2 (4d23h ago)   5d3h
pod/triton-server-7c8965976d-cw5qf  1/1     Running    0               17m

NAME                                                  STATUS   VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/grafana-storage                 Bound    pvc-446cee9d-995d-447e-8505-e5718fce22aa   1Gi        RWO            standard       <unset>                 5d3h
persistentvolumeclaim/prometheus-storage-prometheus-0 Bound    pvc-cf38eacc-a090-4de5-a5a4-55ae2953ebb0   2Gi        RWO            standard       <unset>                 21d
Namespace:              inference-monitoring
Resource                Used    Hard
--------                ----    ----
limits.cpu              3200m   8
limits.memory           2944Mi  16Gi
persistentvolumeclaims  2       5
pods                    6       15
requests.cpu            650m    4
requests.memory         768Mi   8Gi
```

Wynik: 5 podów `Running 1/1`, oba PVC `Bound`, zużycie quota poniżej limitów.

Test 2: audyt bezpieczeństwa (RBAC, NetworkPolicy, non-root)

Cel: uprawnienia najmniejszego zakresu i bezpieczne kontenery (Zadanie 2, Test 2).

```
./scripts/verify_security.sh
```

Test 2: audyt bezpieczeństwa

```
$ ./scripts/verify_security.sh
============================================================
AUDYT BEZPIECZEŃSTWA KUBERNETES DLA CI/CD
Namespace: inference-monitoring
============================================================

[1] Weryfikacja RBAC (Zasada najmniejszych uprawnień - Least Privilege)...
Czy prometheus-sa może czytać pody? (Oczekiwane: yes):
yes
Czy prometheus-sa może tworzyć lub usuwać pody? (Oczekiwane: no):
no
Czy prometheus-sa może czytać Secrets? (Oczekiwane: no):
no

[2] Weryfikacja SecurityContext w podach (Non-root, Capabilities drop)...
Pod: alertmanager-7448b9bdc-kjdrj
  NonRoot: true
  User: 65534
Pod: grafana-db86b4cc6-bdwvp
  NonRoot: true
  User: 472
Pod: loadgen-vgztt
  NonRoot: true
  User: 10001
Pod: mcp-agent-5958656d45-mz77k
  NonRoot: true
  User: 10001
Pod: prometheus-0
  NonRoot: true
  User: 65534
Pod: triton-server-7c8965976d-cw5qf
  NonRoot: true
  User: 10001

[3] Weryfikacja NetworkPolicies (Zero Trust)...
NAME                            POD-SELECTOR                                    AGE
allow-alertmanager              app.kubernetes.io/name=alertmanager             5d3h
allow-grafana                   app.kubernetes.io/name=grafana                  21d
allow-kube-dns                  <none>                                          21d
allow-loadgen-to-prometheus     app.kubernetes.io/name=prometheus               5d1h
allow-mcp-agent                 app.kubernetes.io/name=mcp-agent                21d
allow-operator-prometheus-scrape app.kubernetes.io/name in (mcp-agent,triton-server)  5d
allow-prometheus-scrape         app.kubernetes.io/name=prometheus               21d
allow-triton-server             app.kubernetes.io/name=triton-server            21d
default-deny-all                <none>                                          21d
loadgen-egress                  app.kubernetes.io/name=loadgen                  5d1h
Czy w repozytorium/klastrze nie ma jawnego hasla Grafany w manifestach? (Oczekiwane: brak trafien)

[4] Weryfikacja ResourceQuota i LimitRange...
NAME                    REQUEST                                                                                      LIMIT                                                                AGE
inference-monitoring-quota   persistentvolumeclaims: 2/5, pods: 5/15, requests.cpu: 550m/4, requests.memory: 640Mi/8Gi   limits.cpu: 2700m/8, limits.memory: 2432Mi/16Gi   21d
NAME                     CREATED AT
inference-monitoring-limits   2026-10-15T15:54:05Z

============================================================
Audyt bezpieczeństwa zakończony.
============================================================
```

Wynik: `prometheus-sa` może czytać pody (`yes`), nie może ich usuwać (`no`) ani czytać Secrets (`no`); wszystkie pody mają `NonRoot: true`; polityki sieciowe, ResourceQuota i LimitRange są aktywne.

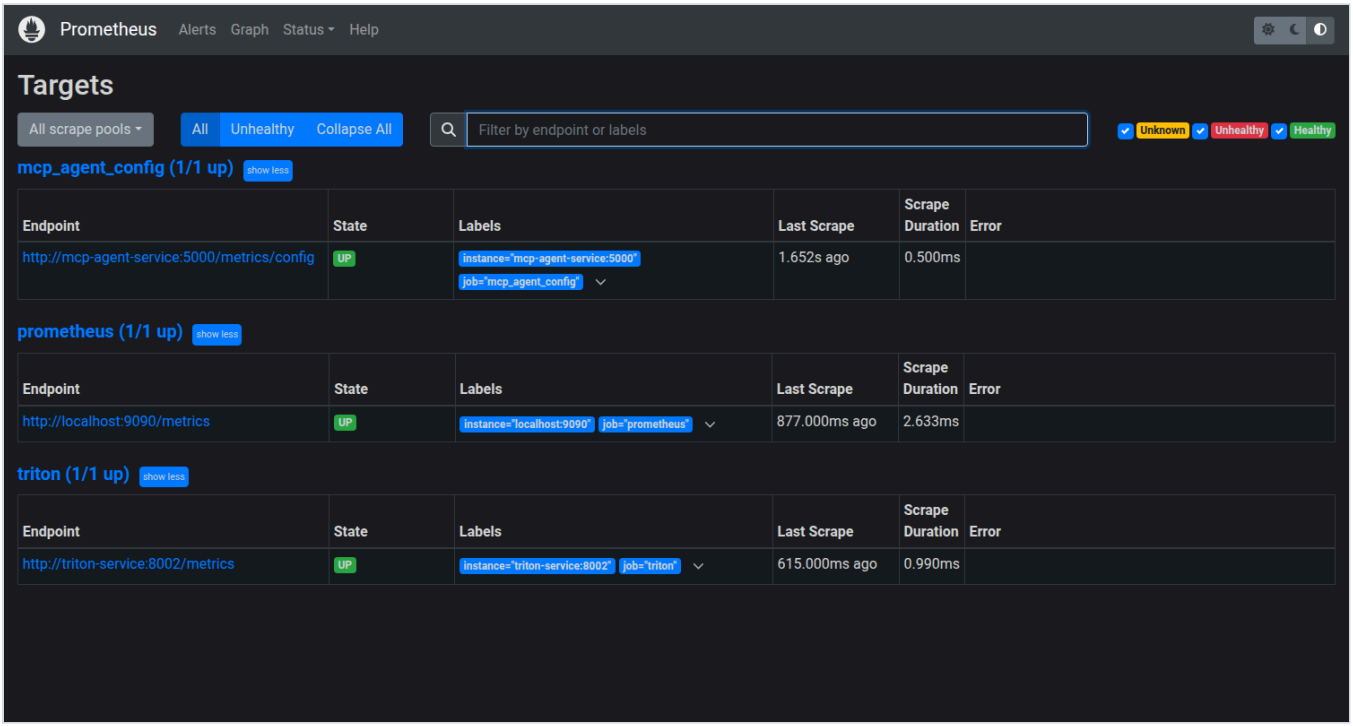
Test 3: potok metryk i wizualizacji

Cel: Prometheus zbiera metryki z Tritona i eksportera, Alertmanager jest podłączony, Grafana ma wczytane źródło danych i dashboard (Zadanie 2, Test 3).

```
curl -s localhost:9090/api/v1/targets | python3 -c "..."      # cele i ich stan
curl -s localhost:9090/api/v1/alertmanagers
```



Interfejs Prometheusa (Status → Targets):



Wynik: Cele `prometheus`, `triton` i `mcp_agent_config` mają stan `up`, Alertmanager jest podłączony.

Testy 4 i 5: skok obciążenia, alert, hot-reload i powrót do normy

Cel: pod wpływem obciążenia rośnie opóźnienie i uruchamia się alert; agent SRE zmienia `max_num_seqs` z 32 na 512 bez przestoju, a opóźnienie wraca do wartości bazowej (Zadanie 2, Testy 4 i 5). Test działa jako Job w klastrze.

```
LOADGEN_IMAGE=<obraz mcp-agent> ./scripts/run_in_cluster_test.sh
```

Testy 4 i 5: skok obciążenia, alert, hot-reload (Job w klastrze)

```
$ LOADGEN_IMAGE=$(kubectl -n inference-monitoring get deploy mcp-agent -o jsonpath='{.spec.template.spec.containers[0].image}') ./scripts/run_in_cluster_test.sh 2>&1 | grep -v REPORT_JSON |
  tail -22
[INFO] Sprawdzanie metryk w Prometheusie...
   [OK] Targety up: 3/3 | reguly alertow: 4 | firing: ['HighInferenceQueueDepth']

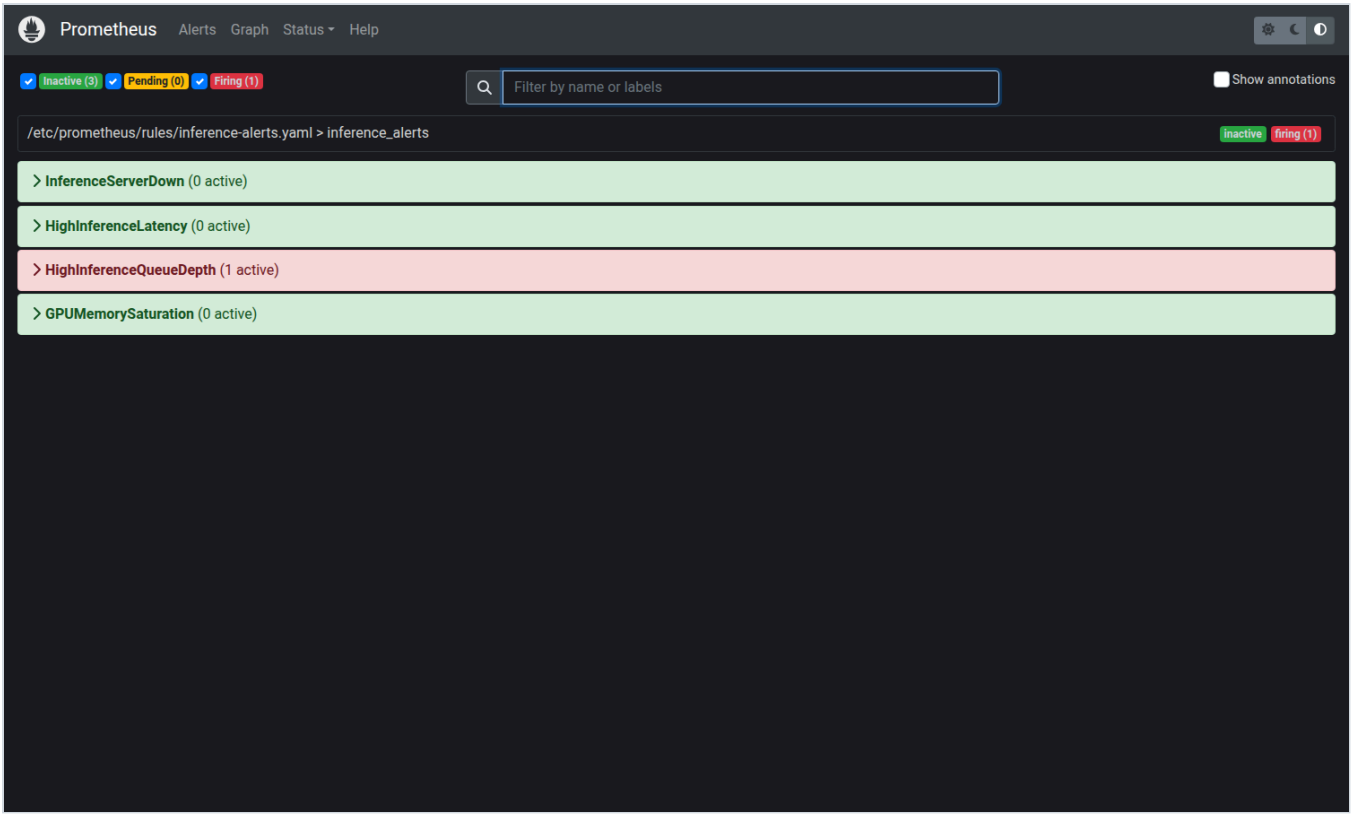
============================================================
PODSUMOWANIE REALIZACJI PROCEDURY TESTOWEJ
============================================================
Faza testu                          | RPS   | Serwer sr.   | Klient sr.   | Klient P95   | Sukces
-------------------------------------------------------------------------------------------------
1. THE CALM (Obciazenie bazowe)       | 20    |    15.79 ms |    26.41 ms |    31.24 ms | 200/200
2. THE STORM (Skok obciazenia - degrad | 400   |   202.08 ms |   634.69 ms |   700.40 ms | 6000/6000
3. THE RESOLUTION (Po rekonfiguracji k | 400   |    14.24 ms |   665.32 ms |   705.16 ms | 6000/6000
============================================================
[PASS] baseline_success_rate>=95%
[PASS] reconfigure_endpoint_200
[PASS] post_reconfig_success_rate>=95%
[PASS] prometheus_targets_up_and_rules_loaded
[PASS] storm_degrades_server_latency>2x_baseline_and_>25ms
[PASS] hot_reload_restores_latency<30ms
[PASS] hot_reload_improves_latency>50%
Raport zapisano do /tmp/test-report.json
Testy zakonczone pomyslnie. Dane zarejestrowane w Prometheusie i Grafanie.
Test procedure PASSED
```

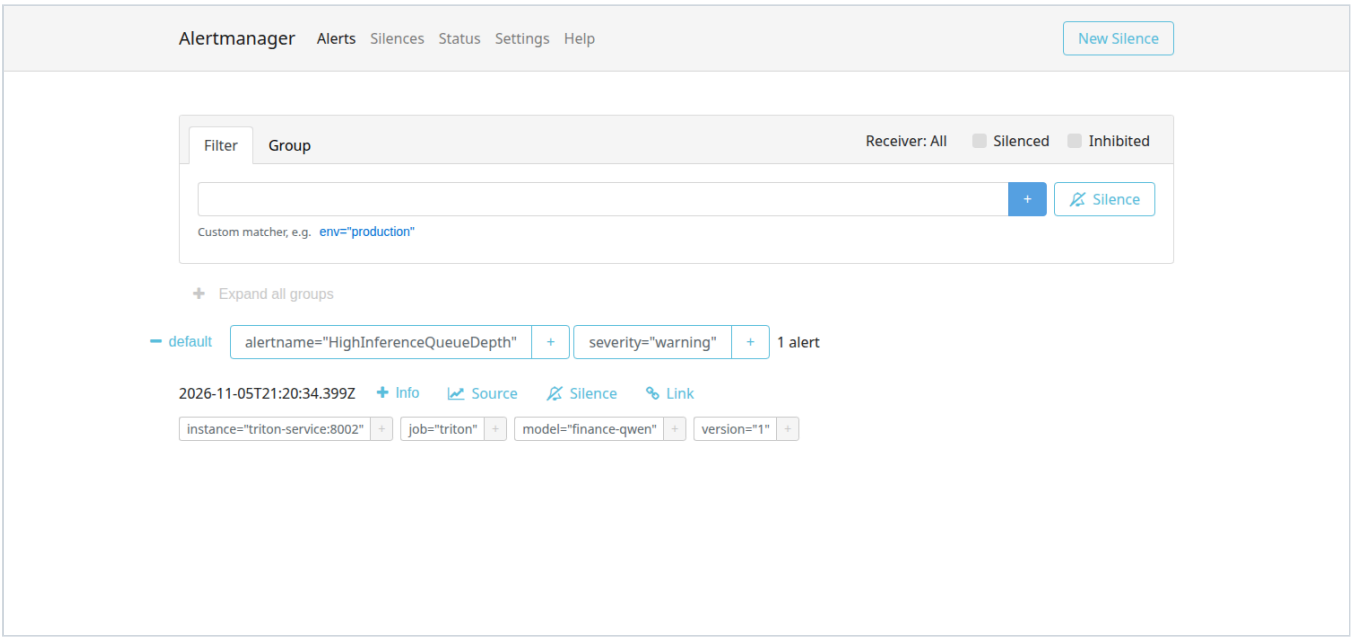
Faza	Żądań/s	Wysłane	Pomyślne	Opóźnienie serwera [ms]
1. CALM	20	200	200	15.8
2. STORM	400	6000	6000	**202.1**
3. RESOLUTION (po hot-reload)	400	6000	6000	**14.2**

Dashboard Grafany po teście (dwa kolejne przebiegi testu: skok ruchu i opóźnienia, następnie powrót do normy; `max_num_seqs` = 512 po hot-reload):



Alerty w Prometheusie i w Alertmanagerze:



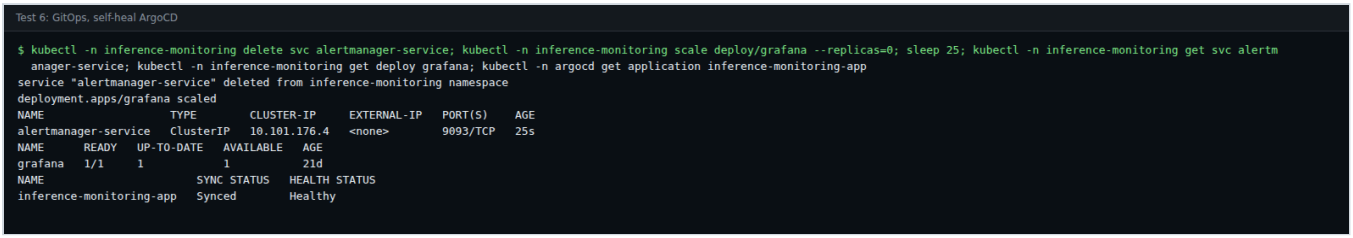


Wynik: wszystkie 7 kryteriów spełnionych. Skuteczność żądań 100%, opóźnienie serwera rośnie w burzy ponad 12.8× względem bazowego i wraca do poziomu bazowego po hot-reload; aktywny alert: HighInferenceQueueDepth. Opóźnienie liczone jest z liczników Prometheusa (`Δ nv_inference_request_duration_us / Δ nv_inference_request_success`), czyli tak samo jak w regule alertu.

Test 6: GitOps, self-heal ArgoCD

Cel: ręczne zmiany w klastrze są cofane do stanu z repozytorium (Zadanie 2, Test 6).

```
kubectl -n inference-monitoring delete svc alertmanager-service
kubectl -n inference-monitoring scale deploy/grafana --replicas=0
sleep 25; kubectl -n inference-monitoring get svc alertmanager-service; kubectl -n inference-
monitoring get deploy grafana
kubectl -n argocd get application inference-monitoring-app
```



Wynik: Usunięty Service i wyzerowane repliki Grafany zostały przywrócone przez ArgoCD, aplikacja ma stan `Synced`.

Test 7: profil Prometheus Operator (ServiceMonitor, Helm)

Cel: Prometheus z chartu `kube-prometheus-stack` wykrywa cele przez ServiceMonitory i ładuje reguły z `PrometheusRule` (Zadanie 2, Test 7).

```
kubectl -n monitoring port-forward svc/kps-prometheus 19090:9090 &
curl -s localhost:19090/api/v1/targets | python3 -c "..."      # cele inference-monitoring
curl -s localhost:19090/api/v1/rules | python3 -c "..."        # grupa inference_alerts
```

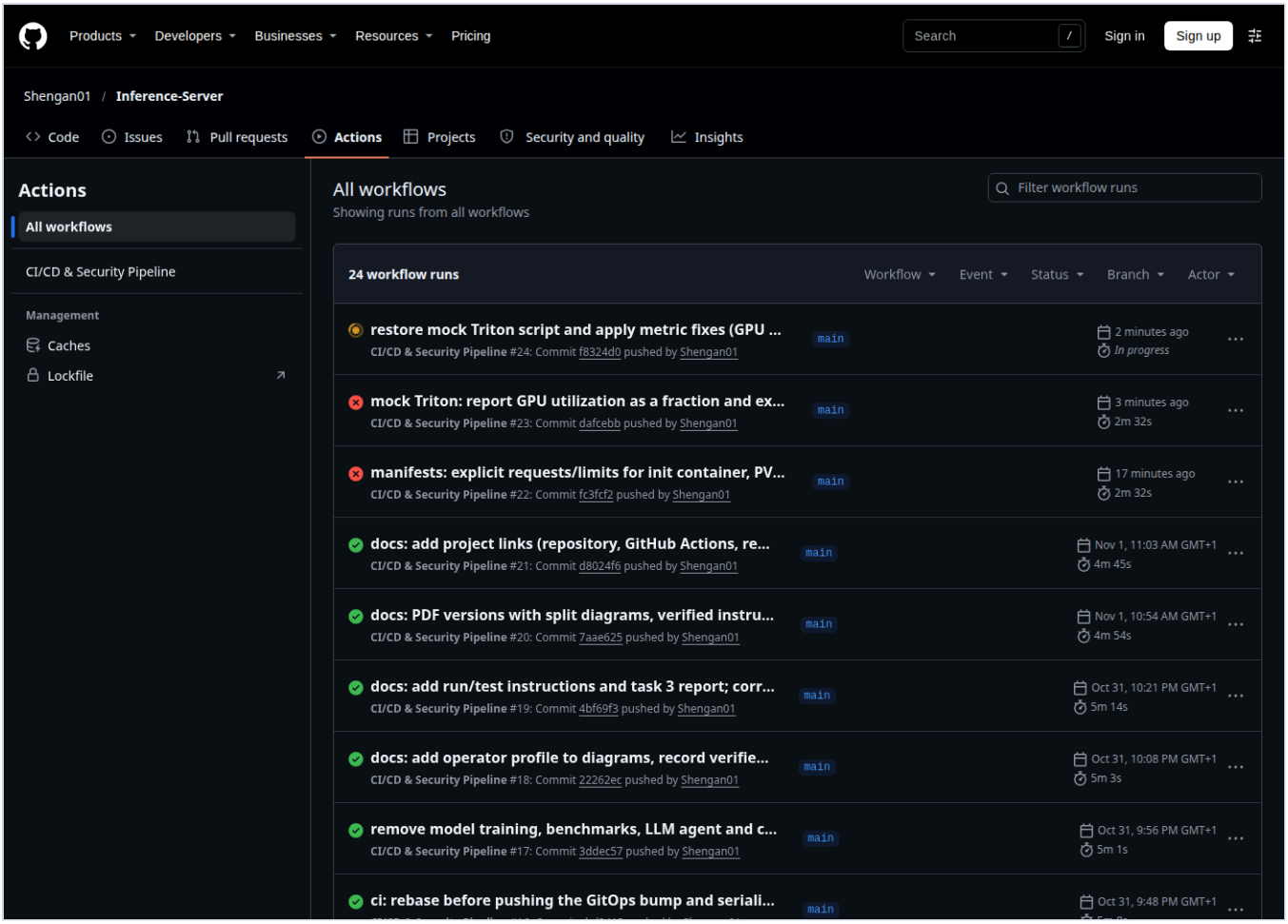


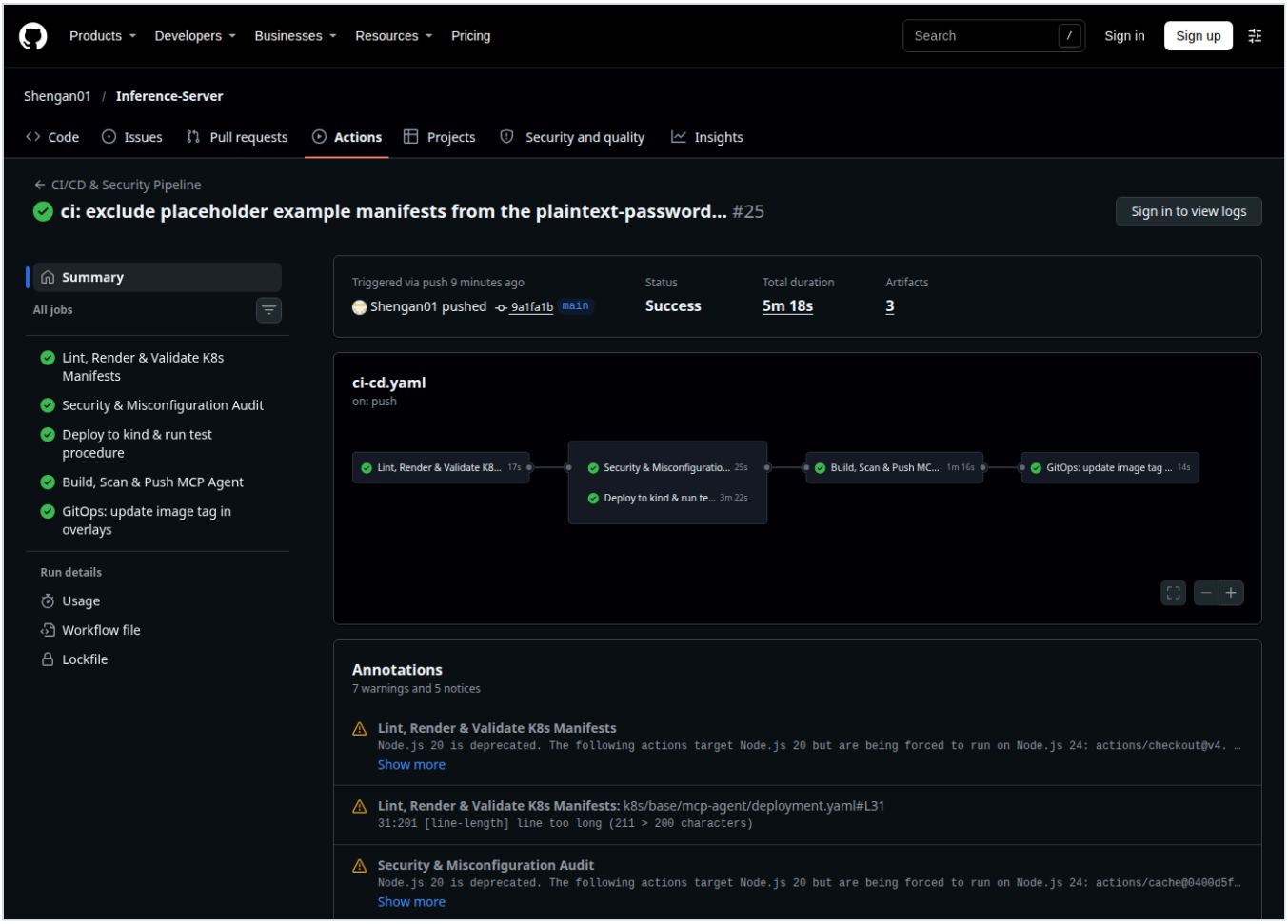
Wynik: Oba ServiceMonitory (`triton`, `mcp-agent-config`) mają stan `up`, grupa `inference_alerts` zawiera 4 reguły.

Test 8: potok CI/CD (GitHub Actions)

Cel: każdy push uruchamia walidację, skan, test w klastrze, zbudowanie obrazu i aktualizację wdrożenia (Zadanie 2, Test 8).

```
https://github.com/Shengan01/Inference-Server/actions
```


Products
Developers
Businesses
Resources
Pricing
Search
/
Sign in
Sign up
Shengan01 / Inference-Server
Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights
Actions
All workflows
CI/CD & Security Pipeline
Management
Caches
Lockfile
All workflows
Showing runs from all workflows
Filter workflow runs
24 workflow runs
Workflow
Event
Status
Branch
Actor
restore mock Triton script and apply metric fixes (GPU ...
CI/CD & Security Pipeline #24: Commit f8324d0 pushed by Shengan01
main
2 minutes ago
In progress
mock Triton: report GPU utilization as a fraction and ex...
CI/CD & Security Pipeline #23: Commit dafcebb pushed by Shengan01
main
3 minutes ago
2m 32s
manifests: explicit requests/limits for init container, PV...
CI/CD & Security Pipeline #22: Commit fc3fcf2 pushed by Shengan01
main
17 minutes ago
2m 32s
docs: add project links (repository, GitHub Actions, re...
CI/CD & Security Pipeline #21: Commit d8024f6 pushed by Shengan01
main
Nov 1, 11:03 AM GMT+1
4m 45s
docs: PDF versions with split diagrams, verified instru...
CI/CD & Security Pipeline #20: Commit 7aae625 pushed by Shengan01
main
Nov 1, 10:54 AM GMT+1
4m 54s
docs: add run/test instructions and task 3 report; corr...
CI/CD & Security Pipeline #19: Commit 4bf69f3 pushed by Shengan01
main
Oct 31, 10:21 PM GMT+1
5m 14s
docs: add operator profile to diagrams, record verifie...
CI/CD & Security Pipeline #18: Commit 22262ec pushed by Shengan01
main
Oct 31, 10:08 PM GMT+1
5m 3s
remove model training, benchmarks, LLM agent and c...
CI/CD & Security Pipeline #17: Commit 3ddec57 pushed by Shengan01
main
Oct 31, 9:56 PM GMT+1
5m 1s
ci: rebase before pushing the GitOps bump and seriali...
main
Oct 31, 9:48 PM GMT+1



Wynik: Przebiegi na gałęzi `main` kończą się sukcesem (5 jobów: walidacja manifestów, skan bezpieczeństwa, test na `kind`, zbudowanie i skan obrazu, aktualizacja tagu w Git).

3. Komentarz do funkcjonalności (zakładana i osiągnięta)

Założenia. Zbudować w Kubernetes system obserwowalności (Prometheus + Grafana + własny eksporter) dla serwera inferencji, z alertami, bezpiecznym dostępem, automatyzacją CI/CD i GitOps. Model nie jest przedmiotem oceny, jest obciążeniem generującym metryki.

Osiągnięte. Wszystkie wymagane obiekty (rozdz. 1.1) są skonfigurowane i działają, a testy 1–8 kończą się wynikiem pozytywnym. System reaguje na przeciążenie (alert), a agent SRE przywraca opóźnienie do wartości bazowej bez przestoju. Dodatkowo wdrożono profil Prometheus Operator, GitOps i potok CI/CD z testem na `kind`.

Ograniczenia (uczciwie): * Wdrożenie i testy wykonano na profilu `cpu-demo`, w którym serwer inferencji jest **mockiem Tritona** (emuluje API i metryki; nie wykorzystuje GPU). Profil `gpu` (prawdziwy Triton z vLLM) nie był jeszcze uruchamiany. Wartości bezwzględne opóźnień zależą od modelu mocka i mocy maszyny; istotny jest kierunek zmiany i powrót do normy. * Opóźnienie po stronie klienta (kilkaset ms przy 400 równoczesnych żądaniach) zależy od mocy generatora i nie opisuje usługi, dlatego w wynikach podano opóźnienie z Prometheusa. * Alertmanager nie ma skonfigurowanego odbiorcy (e-mail/webhook); alerty są widoczne w jego interfejsie. Reguła `HighInferenceLatency` nie jest wymagana przez żaden test; w teście aktywny jest `HighInferenceQueueDepth`. * TLS jest self-signed, a sekrety tworzone są skryptem (poza Git). W środowisku produkcyjnym należy użyć cert-manager i Sealed/External Secrets. * W Minikube kontroler Ingress jest usługą `NodePort` pod `minikube ip` (w chmurze otrzymałby adres

LoadBalancera). * Podczas wykonywania zrzutów wykryto i poprawiono niezgodność mocka z prawdziwym Triton (`nv_gpu_utilization` jako ułamek 0–1 oraz brakująca metryka `nv_inference_pending_request_count`), która powodowała błędne wartości na dashboardzie.

Część B

4. Repozytorium z manifestami

Repozytorium: https://github.com/Shengan01/Inference-Server (publiczne, gałąź `main`)

Plik `README.md` w repozytorium opisuje architekturę, szybki start, strukturę katalogów, metryki i alerty oraz zawiera spis dokumentacji (w tym instrukcję uruchomienia i testów oraz wersje PDF w `docs/pdf/`).

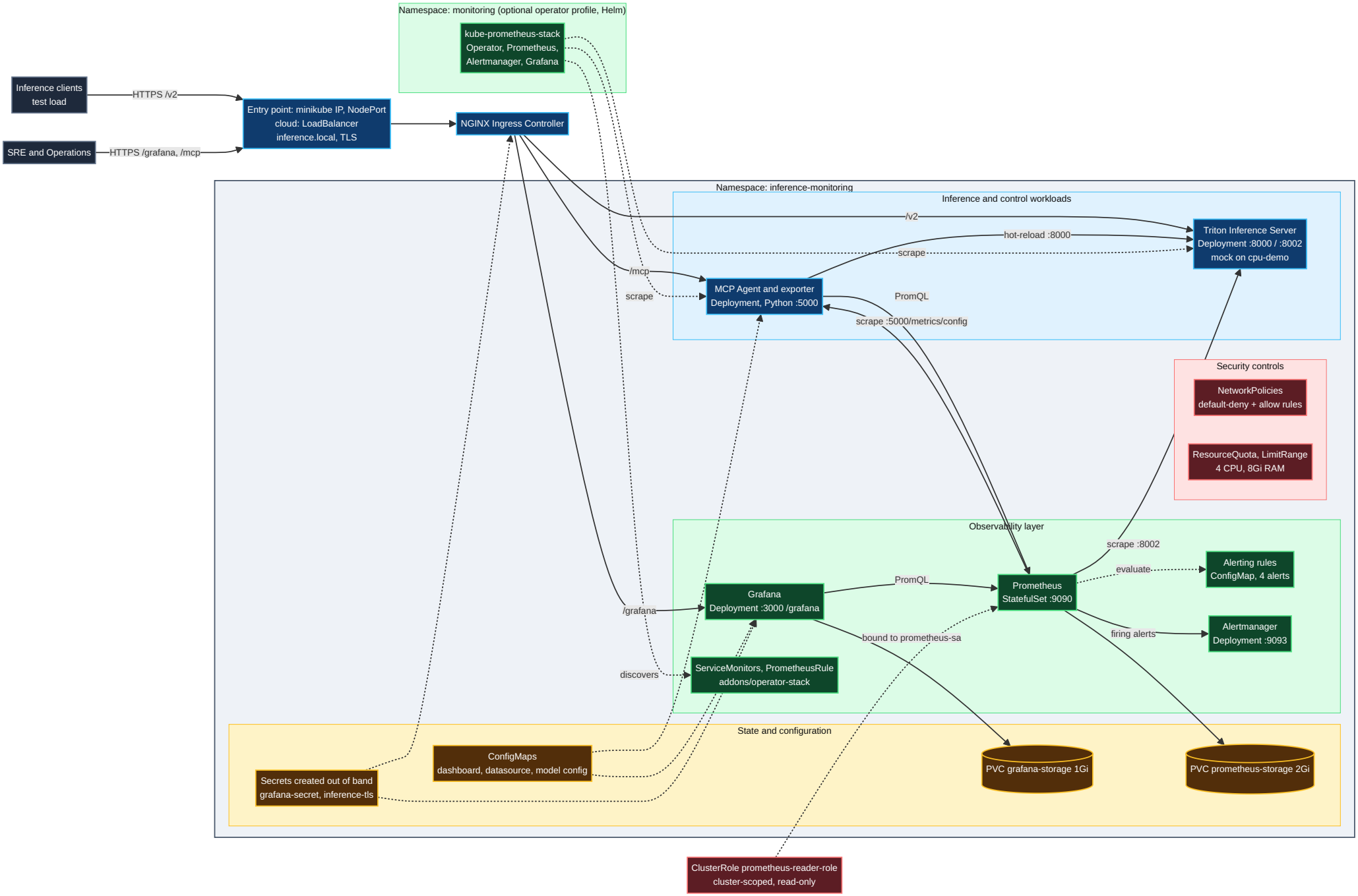
Ścieżka	Zawartość
`k8s/base/`	wspólne manifesty: namespace, RBAC, ResourceQuota/LimitRange, NetworkPolicy, Prometheus (StatefulSet), Grafana, Alertmanager, Ingress, agent
`k8s/overlays/`	profile Kustomize: `cpu-demo` , `gpu` , `local-cpu` , `local-gpu`
`k8s/addons/operator-stack/`	ServiceMonitory, PrometheusRule, NetworkPolicy i wartości Helm (profil operatora)
`k8s/gitops/`	aplikacje ArgoCD
`k8s/tests/` , `k8s/examples/`	Job generatora obciążenia z NetworkPolicy; wzorcowe manifesty Secret
`mcp_agent/`	własny eksporter metryk i kontroler (Python, FastMCP + FastAPI)
`scripts/`	wdrożenie, sekrety, audyt bezpieczeństwa, test w klastrze, zrzuty, PDF
`.github/workflows/ci-cd.yaml`	potok CI/CD
`docs/`	dokumentacja i sprawozdania (Zadanie 2 i 3, dokumentacja techniczna, instrukcja) oraz `docs/pdf/`

Odtworzenie wdrożenia (szczegóły w instrukcji):

```
git clone https://github.com/Shengan01/Inference-Server.git && cd Inference-Server
./scripts/deploy_minikube.sh cpu-demo
LOADGEN_IMAGE=mcp-agent:local ./scripts/run_in_cluster_test.sh
```

Załącznik: architektura systemu

Architektura klastra:



Dostarczanie: CI/CD i GitOps:

